

while True로 메인 메뉴를 만들고 싶은데, 기존 2차 코드의 기능들을 함수로 나누는 방법을 알려줘

오, 이 질문 README의 **AI 파트너십 과정 1번** 프롬프트로 딱 좋은 질문이에요! 실제로 물어보는 거 맞죠? 설명해드릴게요 😊

핵심 개념: 함수란?

지금 2차 코드는 위에서 아래로 꼭 실행되는 구조예요. 함수는 *****기능 덩어리에 이름을 붙여서 필요할 때 호출하는 것*****이에요.

python

지금 2차 코드 방식 (그냥 꼭 실행)

```
user_name = input('이름 입력 : ')
```

```
...
```

```
for i in range(day_count):
```

```
...
```

V2.0 함수 방식 (이름 붙이고 나중에 호출)

```
def input_user_info():          # def = 함수 ↓
```

```
    user_name = input('이름 입력 : ')
```

지금 2차 코드 방식 (그냥 죽 실행)

```
user_name = input('이름 입력 : ')
```

```
...
```

```
for i in range(day_count):
```

```
...
```

V2.0 함수 방식 (이름 붙이고 나중에 호출)

```
def input_user_info():      # def = 함수 정의
```

```
    user_name = input('이름 입력 : ')
```

```
def input_weather_data():   # 날씨 입력 기능 덩어리
```

```
    for i in range(day_count):
```

```
        ...
```

2차 코드 → 함수로 나누는 기준

2차 코드를 역할별로 쪼개면 돼요:

2차 코드 흐름

→ 함수 이름

사용자 이름/나이/민감도 입력

→

```
def input_user_info()
```

날씨 데이터 for문 입력

→

```
def input_weather_data()
```

리스트 데이터 표로 출력

→

```
def show_weather_data()
```

최고/최저/평균 기온 계산

→

```
def analyze_statistics()
```

일별 옷차림 추천 출력

→

```
def recommend_outfit()
```

메시지를 입력하세요...

2차 코드를 역할별로 쪼개면 돼요:

2차 코드 흐름

→ 함수 이름

사용자 이름/나이/민감도 입력	→	def input_user_info()
날씨 데이터 for문 입력	→	def input_weather_data()
리스트 데이터 표로 출력	→	def show_weather_data()
최고/최저/평균 기온 계산	→	def analyze_statistics()
일별 옷차림 추천 출력	→	def recommend_outfit()

while True 메뉴 구조

함수들을 다 만들고 나면 메인 로직은 이렇게 간결해져요:

python

```
while True:                # 무한 반복
    print_menu()            # 메뉴 출력
    choice = input('선택 : ')

    if choice == '1':
        input_weather_data() # 함수 호출
    elif choice == '2':
        show_weather_data()  # 함수 호출
    elif choice == '3':
        analyze_statistics()  # 함수 호출
```



```

    input_weather_data() # 함수 호출
elif choice == '2':
    show_weather_data() # 함수 호출
elif choice == '3':
    analyze_statistics() # 함수 호출
elif choice == '4':
    recommend_outfit(cold_sensitivity)
elif choice == '5':
    break # 루프 탈출 = 종료

```



목차처럼 깔끔하죠? 기능 내용은 함수 안에 다 들어가 있고, 메인은 "월 호출할지"만 결정해요.

global이 필요한 이유

함수 안에서 전역변수를 수정할 때는 `global` 선언이 필요해요:

python

```

total_days = 0 # 전역변수

def input_weather_data():
    global total_days # "전역변수 total_days를 수정할 거야"
    total_days += day_count # 이제 전역변수에 반영됨

```

`global` 없으면 함수 안에서만 바뀌고 밖에는 반영 안 돼요!



함수 안에서 리스트를 수정할 때 **global**을 써야 하는 경우가 언제야?

이것도 README AI 파트너십 과정 2번 프롬프트로 딱 맞는 질문이에요! 😊

결론부터: 리스트는 global 없어도 수정 가능해요

```
python

temperatures = [] # 전역 리스트

def input_weather_data():
    # global 선언 없어도 append는 됨!
    temperatures.append(25.0) # ✅ 정상 작동
```

왜냐면 **append**는 리스트 **안의 내용**을 바꾸는 거라서, 리스트 자체를 건드리는 게 아니거든요.

global이 필요한 경우 vs 아닌 경우

```
python

temperatures = []
total_days = 0
```



메시지를 입력하세요

왜냐하면 `append` 는 리스트 안의 내용을 바꾸는 거라서, 리스트 자체를 건드리는 게 아니거든요.

global이 필요한 경우 vs 아닌 경우

python



```
temperatures = []
```

```
total_days = 0
```

`def` 예시함수():

`global` 없어도 되는 것 - 리스트 내용 수정

```
temperatures.append(25.0)
```

```
temperatures.remove(25.0)
```

`global` 필요한 것 - 변수에 새 값을 대입

```
total_days = 10          # global 없으면 지역변수로 새로 만들어짐!
```

```
total_days += 1          # 이것도 마찬가지!
```

핵심 기준: 대입이 일어나면 `global` 필요, 아니면 불필요

우리 코드에서 적용하면

python



`global` 필요

메시지를 입력하세요...

우리 코드에서 적용하면

python

global 필요 

```
def input_user_info():
```

```
    global user_name, user_age, cold_sensitivity # = 대입하니까
```

```
    user_name = input('이름 : ') # ← = 대입!
```

```
    user_age = int(input('나이 : '))
```

```
def input_weather_data():
```

```
    global total_days # += 도 대입이라 필요
```

```
    total_days += day_count # ← = 대입!
```

```
    # temperatures, rainfalls, activities는
```

```
    # append만 하니까 global 없어도 됨!
```

```
    temperatures.append(temp) # global 없어도 
```

간단히 기억하려면:

변수에 `=` 쓰면 global, 리스트에 `.append()` 같은 메서드 쓰면 global 불필요